University of Nottingham
UK | CHINA | MALAYSIA

EEEE1032 & EEEE1042 – C Programming

Coursework 1 Guidelines

General instructions

1. Complete both programming questions. Each question carries 50 marks.
2. Submit two separate C source files named Q1.c and Q2.c. Use these exact names.
3. Both programs must compile and execute with the C compiler provided with Code::Blocks.
4. Each program must read from standard input in the exact order and format described. Do not hardcode the example data or results.
5. Each program must use at least one array and at least one meaningful user-defined function other than main().
6. You may choose your own algorithms, variable names, data structures, and program organization.
7. Print only the required output records, in the required order. Extra prompts or explanations are not allowed.
8. Only the two C source files are required. No written report is required.
9. The submission deadline is **10 November 2026**.

Question 1. Motor Protection and Monitoring System (50 marks)

An industrial electric motor is equipped with temperature and current sensors. Sensor readings are transmitted to a monitoring system that detects unsafe operating conditions. Develop a C program that processes a sequence of readings, identifies unsafe conditions, and determines when the motor protection system should be activated or reset.

System operation

A reading is unsafe when at least one of the following conditions is satisfied:

- Motor temperature is greater than or equal to 80 °C.
- Motor current is greater than or equal to 8 A.

The protection system initially operates in its normal mode. For an unsafe reading, the system reports WARNING unless that reading causes a trip. To prevent unnecessary shutdowns, the system enters the TRIPPED state only after three consecutive unsafe readings. Once TRIPPED, the system remains in that state until a valid reset occurs. A reading is safe for reset only when BOTH conditions are satisfied:

- Motor temperature is less than or equal to 70 °C.
- Motor current is less than or equal to 6 A.

A reset is permitted only when at least two consecutive readings are safe for reset, including the current reading, AND the current command is RESET. Otherwise, the reset command is ignored.

Additional rules

1. A reading that is neither unsafe nor safe for reset does not count toward either consecutive-reading requirement.
2. The respective consecutive-reading count restarts whenever its sequence is interrupted.
3. While TRIPPED, the output remains TRIPPED regardless of the readings unless a valid reset occurs.
4. Following a valid reset, the system returns to normal operation, and both counts restart.
5. A RESET command received while the system is not TRIPPED is ignored.
6. The reading that causes a trip is reported as TRIPPED. The reading that completes a valid reset is reported as NORMAL.

Dr. Marwan Nafea

7. An unsafe reading that has not yet caused a trip is reported as WARNING. Otherwise, normal operation is reported as NORMAL.
8. Count each transition into TRIPPED as one trip and each valid reset as one reset.

Input and output

Input: The first line contains the integer N, followed by N lines, each containing the temperature, current, and command, in that order. Temperature and current measurements may contain decimal values. The corresponding parameters and their constraints are summarized in Table 1.

Table 1. Input parameters and constraints for the motor protection and monitoring system.

Item	Definition and constraints
N	Number of sensor readings; $1 \leq N \leq 200$
Temperature	Motor temperature in °C; $-20 \leq$ temperature ≤ 150
Current	Motor current in A; $0 \leq$ current ≤ 20
Command	Either NONE or RESET (uppercase)

For each reading, print its sequence number (starting at 1), followed by NORMAL, WARNING, or TRIPPED. After processing all readings, print the total number of trips and successful resets. Your program must work correctly with other valid reading sequences, including repeated trips and resets.

Example input

```
8
72.5 4.2 NONE
82.3 5.4 NONE
79.6 8.0 NONE
80.0 7.1 NONE
75.5 7.0 RESET
69.5 5.4 NONE
70.0 6.0 RESET
81.2 9.1 NONE
```

Expected output

```
1 NORMAL
2 WARNING
3 WARNING
4 TRIPPED
5 TRIPPED
6 TRIPPED
7 NORMAL
8 WARNING
TRIPS 1 RESETS 1
```

Question 2. Electromechanical Actuator Selection (50 marks)

An engineer is developing an automated robotic mechanism with different torque and rotational-speed requirements. Several motor–gearbox combinations are available, each with different operating characteristics and limits. Develop a C program that evaluates the available combinations and selects the most energy-efficient candidate capable of completing an entire operating cycle.

Engineering model

An operating cycle contains several stages. Each stage specifies a load torque, rotational speed, and duration. For each candidate, use the simplified model presented in Table 2.

Table 2. Engineering equations for electromechanical actuator evaluation.

Quantity	**Equation**
Required motor torque (Nm)	$T_m = \frac{\lvert T_L \rvert}{G\eta}$
Required motor speed (rpm)	$n_m = G\lvert n_L \rvert$
Estimated motor current (A)	$I = I_0 + \frac{T_m}{K_t}$
Electrical energy per stage (J)	$E_j = VI_j\Delta t_j$
Total cycle energy (J)	$E_{total} = \sum_{j=1}^{S} E_j$

In Table 2, T_L is the load torque, G is the gearbox ratio, η is the gearbox efficiency as a decimal, n_L is the load speed, I_0 is the no-load current, K_t is the motor torque constant, V is the supply voltage, Δt_j is the stage duration, and S is the number of operating stages. Use the magnitudes of torque and speed for both positive and negative directions. Assume identical motor behavior in either direction, and ignore regenerative braking, acceleration transients, and thermal effects.

Feasibility and selection

A candidate is FEASIBLE only if ALL of the following conditions hold for EVERY operating stage:

1. Required motor torque does not exceed the candidate's maximum motor torque.
2. Required motor speed does not exceed the candidate's maximum motor speed.
3. Estimated motor current does not exceed the maximum allowable supply current.

A candidate that violates even one limit in one stage is INFEASIBLE. Equality with a permitted maximum is acceptable. Among feasible candidates, choose the candidate with the lowest total electrical energy. If candidates have exactly equal unrounded energies, choose the identifier that comes first alphabetically. If no candidate is feasible, report BEST NONE.

Input format

The first line contains S M. The second line contains V I_{max}. The next S lines contain load_torque load_speed duration. The final M lines contain ID G efficiency T_{max} n_{max} K_t I_0. All numerical inputs are finite and sufficiently small for the required calculations to remain within the range of C double. Table 3 summarizes the input parameters and their constraints for electromechanical actuator selection.

Table 3. Input parameters and constraints for electromechanical actuator selection.

Item	Definition and constraints
S, M	$1 \le S \le 100$ stages; $1 \le M \le 50$ candidates
V, I_{max}	$V > 0$ and $I_{max} > 0$
Stage data	Load torque and speed may be positive, negative, or zero; duration > 0
ID	Unique identifier: 1–12 uppercase letters or digits, starting with a letter
G, η	$G > 0$; $0 < \eta \le 1$
T_{max}, n_{max}	$T_{max} \ge 0$; $n_{max} \ge 0$
K_t, I_0	$K_t > 0$; $I_0 \ge 0$

Required output

For each candidate, in the original input order, print ID FEASIBLE energy or ID INFEASIBLE. Display feasible candidates' energy in joules with exactly two decimal places. Finally, print BEST ID energy for the selected candidate, or BEST NONE when there is no feasible candidate. Make feasibility and selection decisions using unrounded values; round only for display. Your program should evaluate all stages and candidates and identify the best feasible solution according to the stated model.

Example input

```
3 3
24 3
4 60 2
-2 -30 3
0 0 1
A1 10 0.8 0.7 800 0.2 0.3
B2 12 0.75 0.5 650 0.25 0.2
C3 15 0.9 0.6 1000 0.3 0.25
```

Expected output

```
A1 FEASIBLE 253.20
B2 INFEASIBLE
C3 FEASIBLE 118.96
BEST C3 118.96
```

Assessment Rubric

Each question carries 50 marks. Your solutions will be assessed against the requirements of the questions, including correctness, reliability, and appropriate use of C programming constructs. Alternative valid implementations are acceptable. The assessment rubric for evaluating both programming questions is presented in Table 4. The performance categories are expressed as percentages and are applied independently to each question, with a maximum of 50 marks per question.

Table 4. Assessment rubric for the C programming coursework.

Category	Assessment Aspect	Mark (%)
Unsatisfactory	The program demonstrates very limited achievement of the coursework requirements. It may fail to compile or execute, contain major syntax or logical errors, or produce substantially incorrect results. Implementation is incomplete, with significant deficiencies in input processing, engineering calculations, decision-making, or program structure.	0–29
Limited	The program demonstrates basic functionality but only partially satisfies the coursework requirements. Important engineering calculations, operating conditions, or decision-making processes are incorrect or incomplete. Outputs may be inconsistent or inaccurate, and the application of programming concepts, arrays, and functions shows significant weaknesses.	30–49
Satisfactory	The program compiles and executes successfully, demonstrating a reasonable understanding of programming concepts and engineering requirements. Most basic calculations, input processing, and decisions are correct, producing acceptable results under common operating conditions. However, noticeable logical errors, incomplete functionality, or inconsistencies remain, particularly in more demanding scenarios.	50–64
Good	The program executes reliably and satisfies most coursework requirements. Engineering calculations, input processing, conditional operations, and decision-making are generally correct. Outputs are accurate under various operating conditions, with occasional errors or limitations. Arrays and user-defined functions are used appropriately, and the program demonstrates good organization, readability, and efficiency.	65–79
Very Good	The program demonstrates a high level of correctness, reliability, and engineering problem-solving competence. Nearly all requirements are satisfied, with accurate calculations, appropriate engineering decisions, and consistent outputs across normal, complex, and boundary conditions. The implementation demonstrates well-structured logic, effective use of arrays and functions, and efficient processing. Only minor weaknesses or opportunities for improvement remain.	80–89
Exceptional	The program demonstrates outstanding correctness, robustness, efficiency, and engineering problem-solving competence. All functional and engineering requirements are satisfied consistently across normal, complex, and boundary conditions. The solution demonstrates precise calculations, reliable input processing, appropriate engineering decisions, and excellent program organization. Arrays and user-defined functions are used effectively, with clear and maintainable code. Marks approaching 100% are reserved for solutions demonstrating exceptional attention to detail, without identifiable functional, logical, or structural deficiencies.	90–100